C-Bounded Powersort's time complexity and correctness analysis

Théo Araújo Magalhães

April 5, 2025

Definitions:

Runs: Contiguous regions of the input that are already in sorted order. Let R_0, ..., R_{r-1} be the runs in the input, L_i denotes the length of R_i.*

Run-adaptive mergesort: Any stable run-adaptive mergesort can be described as a merge tree T: an (ordered rooted) tree with leaves R_0, ..., R_{r-1} (appearing in that order from left to right). Each internal node v of T corresponds to the result of merging its children.*

Merge costs: M(v) is the total length of the resulting merge corresponding to the internal node v. The total merge cost of T is then:

$$M(T) = \sum_{v \in T} M(v). \tag{1}$$

Run boundaries: Denoted by B_i, it is the boundary between the (i - 1)st and ith run (i=1, ..., r - 1). In binary merge trees, there is a one-to-one correspondence between the B_i and the (inner/non-leaf) merge-tree nodes.*

Node depth: Let d_i be the depth of leaf R_i in T, where depth is the number of edges on the path to the root. Every element in run R_i is counted exactly d_i times in $\sum_v$M(v), so:

$$M = \sum_{i=0}^{r-1} d_i \cdot L_i * \tag{2}$$

Node power: Let $\alpha_0, \ldots, \alpha_m, \sum \alpha_j = 1$ be leaf probabilities. For 1 ≤ j ≤ m, let j be the internal node separating the (j - 1)st and jth leaf. The power of (the split at) node j is:

$$P_j = min\{\ell \in N : \lfloor \frac{a}{2^{-\ell}} \rfloor < \lfloor \frac{b}{2^{-\ell}} \rfloor\}, \text{where } a = \sum_{i=0}^{j-1} \alpha_i - \frac{1}{2}\alpha_{j-1}, b = \sum_{i=0}^{j-1} \alpha_i + \frac{1}{2}\alpha_j. \tag{3}$$

(Pj is the index of the first bit where the (binary) fractional parts of a and b differ.)

Powersort: The intuition behind Powersort is to imagine a complete binary tree T_v sitting on top of the array, where each element A[i] of the input corresponds to a leaf of T_v .T_v corresponds to the merge tree of a non-adaptive two-way Mergesort (starting with individual elements). Now each run boundary B_j also corresponds to

an (inner) node w of this (virtual) k-ary tree TV , namely the lowest common ancestor of the two leafs adjacent to B_j.* [We need to re-explain this part better so that we can talk about the concept of a "Powersort merge tree" or something of the sorts. Helps with the proof of our main proposition]

Forgetful K-Stack: A forgetful stack is a stack that forgets its k-th element if it already has k elements in it and a new one is added.

C-Bounded Powersort: The Powersort algorithm using a Forgetful C-Stack instead of the traditional stack with size logn + 1.

Processing a subtree of height h: Let T be a merge tree, T' one of its subtrees of size h and v the root of T'. Processing T' is executing all the merges necessary to fill node v, given that it has length M(v).

Proofs:

Proposition: Let T be a Powersort merge tree of height H. One pass of the C-Bounded Powersort algorithm is enough to process all subtrees of T with height of C-1.

Proof by induction:

Base case - T of height 2:

Let v be the root of T and v_1 and v_2 its two children. Therefore, T has two subtrees T' a T" of size one. For the left subtree T', v_1 is the root and it has only two leaf children: the runs R_1 and R_2. Given that h = 1, a Forgetful 2-Stack will work with 2-Bounded Powersort as follows:

R_1 will be found and added to the stack, then, given that there is nothing to compare R_1 with, the algorithm will search for the adjacent run, R_2. After finding it, the algorithm will calculate the power between R_1 and R_2, which is equal to the height of v_1. Given that we only have one power calculated, this will lead to no merges and R_2 will be added to our stack. Afterwards, the algorithm will proceed to the right T " subtree, which is rooted in v_2. The algorithm will find R_3 and, then, calculate the power between R_2 and R_3. This will result in the height of v, which is smaller than that of v_1, our previously calculated power, leading to the merge of runs R_1 and R_2 into run R_{12}.

After this, our Forgetful 2-Stack will contain R_{12} and R_3. Then, the algorithm will find run R_4. Calculating the power between R_3 and R_4 leads to height v_2, which is bigger than that of v, our previously calculated power, meaning the algorithm will not merge any runs and will add R_4 to the Forgetful 2-Stack. This will cause the stack to "forget" about R_{12}, meaning it will only contain R_3 and R_4. Following the logic of what the calculation of the power between to runs means, the power between R_3 and R_4 is higher than that of R_{12} and R_3, therefore, the merge between R_3 and R_4 has to happen before that of R_{12} and R_3, so, the algorithm will merge R_3 and R_4, leading to the processing of both subtrees of T' and T".

[Comentário em português: Uma das coisas que ficou aberta no entendimento do algoritmo modificado foi o que fazer quando alcançamos o final do vetor. Acho que a ideia é dar merge para trás até alcançarmos a altura do último merge que aconteceu

antes do fim do vetor. No caso de que nenhum merge aconteceu, podemos dar merge para trás subindo C-1 níveis.]

Hypothesis: Suppose the proposition holds true for all subtrees of height h ≤ k in a powersort merge tree.

Inductive Step: Let T be a Powersort merge tree of height H. We will prove that a (k+2)-Bounded Powersort processes all of T's subtrees that have height of h = k+1. Let T' be one of those subtrees and v its root. We will analyze the sub-trees T_ℓ and T_r, each rooted in the left and right children of v, v_ℓ being the left child and v_r the right child. The height of the T_ℓ is, at most, k, therefore, we can apply our induction hypothesis and process T_ℓ, thus, v_ℓ will contain run R_ℓ, which is the result of the merge of all runs in T_ℓ. The processing of this sub-tree will happen once the most-left run of T_r is analyzed by the algorithm.[1] By the end of the processing of T_ℓ, our Forgetful (k+2)-Stack will contain R_ℓ and the most-left run of T_r. Given that, by hypothesis, the processing of T_r requires one pass of a (k+1)-Bounded Powersort[2][3], and that our Forgetful (k+2)-Stack has exactly k spaces left, with one of them already being occupied by one run of T_r, the algorithm will be able to process T_r. Similarly to what happened during the processing of T_ℓ, the calculation of the power between the right-most run of T_r (which is also the right-most run of T') and the next run, which is the left-most run of another subtree of T with height k+1, will result in the height of v - 1.[4]. Given that all the powers calculated between runs in T' are greater than v-1, the algorithm will merge all the remaining runs in our Forgetful (k+2) stack, leading to the processing of T_r but also to the processing of T', given that R_ℓ will still be on the stack alongside all runs from T_r. Therefore, if the (k+2)-Bounded Powersort was able to process a generic T' subtree of T with height k+1, it is fair to assume that it will be able to further process all the remaining subtrees of same height by following the same logic as the one described for T'. □

[Comentário em português: Seria, aqui, necessário, explicitar novamente como funcionaria o algoritmo no final do vetor? Já está no caso base, mas...]

[1] Once the power between the most-right run of T_ℓ with most-left run of T_r is calculated, it will result in the height of v, which is smaller than the height of v_ℓ, meaning the algorithm will merge everything left in our Forgetful k+2-Stack, for all powers calculated between runs in T_ℓ is greater than v_ℓ

[2] Remember that T_ℓ and T_r have, at max, height of k

[3] Requiring a (k+1)-Bounded Powersort means the processing needs k+1 free spaces in the Forgetful C-Stack, not a specific tuning of the algorithm.

[4] This is due to the fact that both runs are in adjacent subtrees: v is the left child of a node v_p and there is a node v' which is the right child of the same v_p, the first common ancestor between v and v' is v_p, therefore, the first common ancestor between runs in each subtree (the subtree rooted in v and the subtree rooted in v') is v_p

Proposition: It takes C-Bounded Powersort at most $\mathcal{O}(n \cdot \log n)$ element scans to order an array of size n.

Proof. Each pass through the array scans element i, with 0 ≤ i ≤ n, at least once. Given that the algorithm will do $\frac{\log n}{c-1}$ passes[5], i will be read at least $\frac{\log n}{c-1}$ times. Once the entire array is ordered, i will also have been read at least twice for each merge it was present in, i.e, $2 \cdot d_i$, with d_i being equal to the height of T. Therefore, the total number of scans is:

$$Scans \leq \sum_{i=0}^{n} 2 \cdot \log n + n \cdot \frac{\log n}{c-1} \tag{4}$$

$$Scans \leq n \log n \cdot (2 + \frac{1}{c-1}) \tag{5}$$

$$Scans \leq \mathcal{O}(n \cdot \log n) \tag{6}$$

□

[5]The height of a Powersort merge tree T is $\log n$ and the C-Bounded Powersort algorithm starts by processing subtrees of T that have height less than or equal to c-1. This results in a smaller tree T' of height $\log n$ - (c-1). A second pass will, once again, process the subtrees of T' that have height less than or equal to c-1, resulting in a smaller tree T''. Therefore, if k is the amount of trees created by this process until the last tree is of height less than or equal to c-1, meaning it will only require one last pass of the algorithm to be processed, k·(c-1) ≤ $\log n$